

Curso de Go

Aula 3 - Tipos e Variáveis em Go

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **código_**^[3]
BRAZUCA





Nessa Aula nos Vamos Ver

- **Tipos primitivos**
- **Inferência de tipos**
- **Escopo de variáveis**
- **Criar um programa na parte prática**



O que são tipos em Go

- **Tipos definem o tipo de dado armazenado**
- **Go é uma linguagem fortemente tipada**
- **Evita erros comuns em tempo de execução**

Tipos:

- **int → números inteiros (ex: 10, -5, 42)**
- **string → textos (ex: "Olá", "Go", "Antonio")**
- **bool → valores booleanos (true ou false)**
- **float64 → números decimais (ex: 3.14, 19.99)**



Tipo int

- **Representa números inteiros**
- **Pode ser positivo ou negativo**
- **Muito usado para contagem e índices**
- **int → números inteiros (ex: 10, -5, 42)**

Exemplo de código

```
package main
```

```
import "fmt"
```

```
func main() {  
    var idade int = 30  
    fmt.Println(idade)  
}
```



Tipo string

- Representa textos
- Sempre entre aspas
- Muito usado para nomes, mensagens, etc
- **string** → textos (ex: "Olá", "Go", "Antonio")

Exemplo de código

```
func main() {  
    var nome string = "Antonio"  
    fmt.Println(nome)  
}
```




Tipo bool

- Representa verdadeiro ou falso
- Muito usado em condições
- **bool** → valores booleanos (true ou false)

Exemplo de código

```
func main() {  
    var ativo bool = true  
    fmt.Println(ativo)  
}
```



Tipo float

- **Representa números decimais**
- **Usamos geralmente float64**
- **float64 → números decimais (ex: 3.14, 19.99)**

Exemplo de código

```
func main() {  
    var preco float64 = 19.99  
    fmt.Println(preco)  
}
```



Inferência de tipos com :=

- O Go descobre o tipo automaticamente
- Só pode ser usado dentro de funções
- Forma mais comum no dia a dia

Exemplo de código

```
func main() {  
    nome := "Antonio"  
    idade := 30  
    ativo := true  
  
    fmt.Println(nome, idade, ativo)  
}
```




Inferência de tipos com `:=`

Comparando formas de declarar

Forma tradicional:

Exemplo de código

```
var idade int = 30
```

Forma Curta

Exemplo de código

```
idade := 30
```

- **Ambas são válidas**
- **`:=` é mais usado em código moderno**



Escopo de variáveis

- **Define onde a variável pode ser usada em um código**
- **Pode ser local ou global**
- **Evita conflitos no código**



Variável local

- Declarada dentro de uma função
- Só existe dentro dela

Exemplo de código

```
func main() {  
    nome := "Antonio"  
    fmt.Println(nome)  
}
```



Variável global

- Declarada fora das funções
- Pode ser usada em todo o programa

Exemplo de código

```
package main
```

```
import "fmt"
```

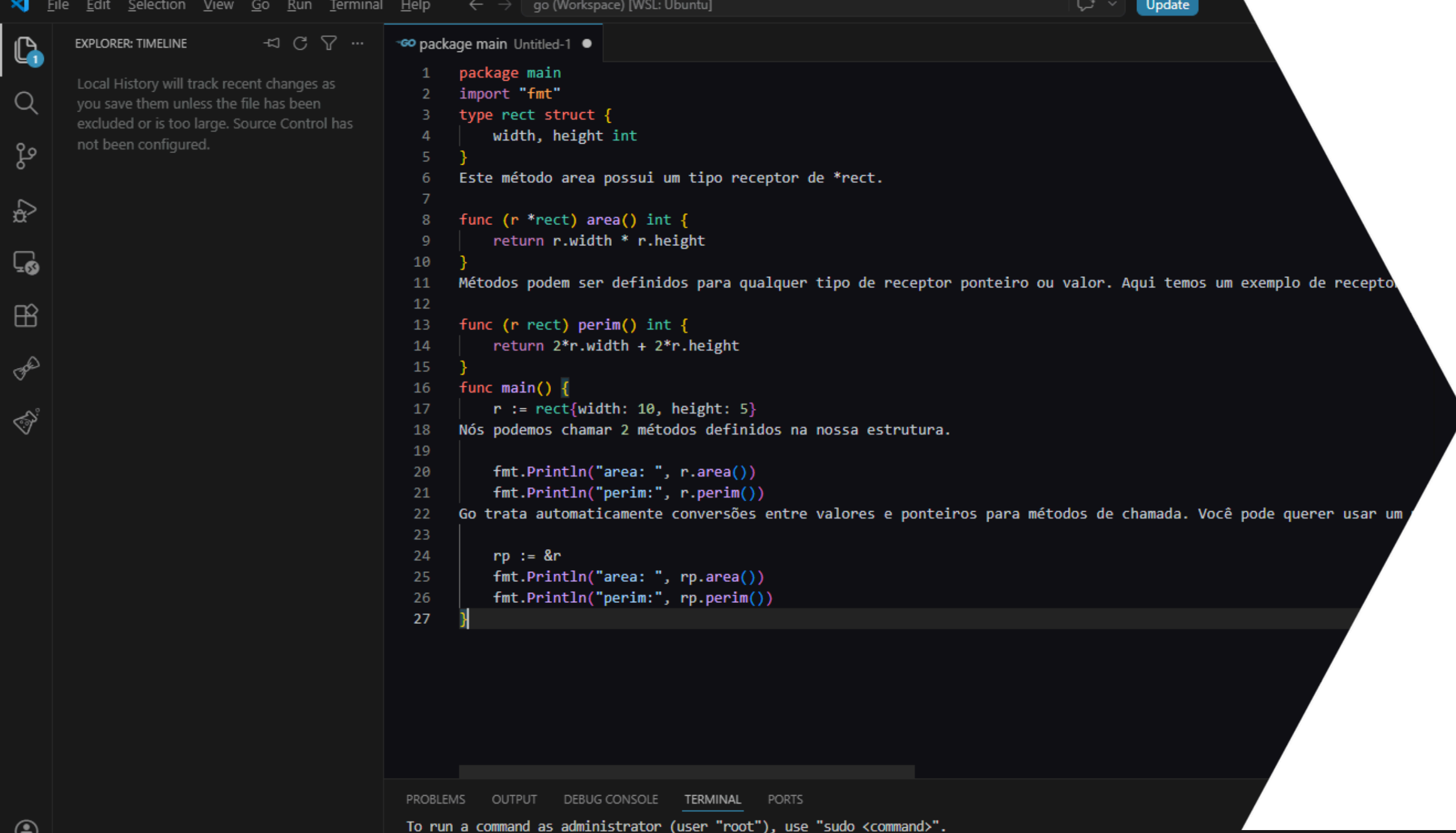
```
var nome string = "Antonio"
```

```
func main() {  
    fmt.Println(nome)  
}
```



Boas práticas

- **Prefira `:=` dentro de funções**
- **Use nomes claros para variáveis**
- **Evite variáveis globais desnecessárias**
- **Mantenha o escopo o menor possível**



```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

Parte Prática

Praticar tipos, inferência e escopo em Go